Machine Learning Term Project Team 4

Final Report

Predicting Yelp Review Star Ratings:
A Comparative Study of Traditional and Deep Learning Methods

Alexander Nardi, Anthony Mahon, Benedetto Falin, Keating Sane, Mohammed Mamdouh

May 4, 2026

Contents

1 Team Information **3**
1.1 Team Members and Roles . . . 3
1.2 Responsibility Distribution . . . 3
1.3 Team Assignment Table . . . 3

2 Problem Setup **4**
2.1 Classification Task . . . 4
2.2 Dataset . . . 4
2.3 Dataset Size and Splits . . . 4
2.4 Input and Output Format . . . 4
2.5 Class Labels . . . 4
2.6 Basic Dataset Statistics . . . 5
2.7 Project Goals . . . 5

3 Implementation Description **5**
3.1 Repository and Codebase Overview . . . 5
3.2 Data Preparation and Experimental Pipeline . . . 6
3.3 Traditional Machine Learning Models . . . 8
3.4 Deep Learning Models . . . 13
3.5 Cross-Model Engineering Choices . . . 20

4 Usage Guide **21**
4.1 Environment Setup . . . 21
4.2 Common Run Modes . . . 21
4.3 How to Run Each Model . . . 22
4.4 Expected Output Files . . . 23

5 Evaluation Metrics **23**
5.1 Notation . . . 23
5.2 Accuracy . . . 24
5.3 Macro Precision . . . 24
5.4 Macro Recall . . . 24
5.5 Macro F1 Score . . . 24
5.6 Confusion Matrix . . . 24
5.7 Why These Metrics Fit This Project . . . 25

6 Results **26**
6.1 Comparison Table 26
6.2 Per-Model Analysis 26
6.3 Hyperparameter Sensitivity 35

7 Conclusions & Future Work **36**
7.1 Key Findings 36
7.2 Limitations 36
7.3 Future Improvements 37
7.4 Final Takeaway 37

1. Team Information

1.1 Team Members and Roles

Alexander Nardi - Project Coordinator / Member
Anthony Mahon - Member
Benedetto Falin - Member
Keating Sane - Member
Mohammed Mamdouh - Member

1.2 Responsibility Distribution

Each team member received one traditional model and one deep learning model. The amount of work that one did was attempted to be balanced based on effort. For example, with an easier traditional model (i.e. linear SVM), you had a more complex deep learning model (i.e LSTM), and vise versa.

All model's made by all members are intended for the same task, as described in Section 2.

1.3 Team Assignment Table

Team Member	Models
Alexander Nardi (Project Coordinator)	Linear SVM, LSTM
Anthony Mahon	Kernel SVM, Transformer encoder
Benedetto Falin	Naive Bayes, Multi Layer Perceptron
Keating Sane	Decision Tree/Random Forest, CNN
Mohammed Mamdouh	Logistic Regression, LLM

2. Problem Setup

2.1 Classification Task

The task is five-class single-label text classification. Given the raw text of a Yelp review, the model must predict the star rating the reviewer assigned, which is one of five ordinal classes ranging from 1 star to 5 stars. Each review belongs to exactly one class. The ordinal structure of the labels means that misclassifying a 1-star review as 2-star is a smaller error than misclassifying it as 5-star, though all standard metrics used here treat all misclassifications equally.

2.2 Dataset

The dataset used is **Yelp Review Full**, a large-scale sentiment classification benchmark compiled from the 2015 Yelp Dataset Challenge. It consists of English-language Yelp business reviews paired with their star ratings. Reviews vary considerably in length, from a single sentence to several paragraphs, and cover a wide range of business categories. The dataset is publicly available through HuggingFace Datasets under the identifier `yelp_review_full`.

2.3 Dataset Size and Splits

Split	Samples	Notes
Training	585,000	90% of the official 650,000-review training partition
Validation	65,000	10% stratified split carved from the training partition
Testing	50,000	Official held-out test partition, never seen during tuning

2.4 Input and Output Format

Input: Raw English review text as a variable-length string. No manual preprocessing is required before passing to the data loader. Sparse feature models (Naive Bayes, MLP, SVM, Logistic Regression, Decision Tree, Random Forest) convert this text to a fixed-size TF-IDF vector via the shared `fit_tfidf_features()` utility. Sequence models tokenize and index the text into fixed-length integer sequences.

Output: A single integer class prediction in the range $\{0, 1, 2, 3, 4\}$, corresponding to star ratings 1 through 5. Labels are zero-indexed internally throughout the codebase; the mapping to human-readable star ratings is applied only at evaluation and display time.

2.5 Class Labels

Label ID	Class Name	Meaning
0	1 star	Very negative review
1	2 stars	Negative review
2	3 stars	Mixed or neutral review
3	4 stars	Positive review
4	5 stars	Very positive review

2.6 Basic Dataset Statistics

The dataset is approximately balanced across all five classes. Each class accounts for exactly 20% of both the training and test partitions, with minor variation. Training class counts are approximately 130,000 reviews per class before the validation split is carved out. This near-uniform distribution means accuracy is a meaningful primary metric and no class weighting or oversampling is required.

Reviews vary substantially in length. Short reviews of one to two sentences are common in the extreme rating classes (1 star and 5 stars), where the reviewer's sentiment is unambiguous and concisely expressed. Middle-rating reviews (2, 3, and 4 stars) tend to be longer and more qualified, mixing positive and negative observations, which contributes to the difficulty of separating these classes.

2.7 Project Goals

The primary goal is to implement and compare ten machine learning models spanning traditional and deep learning approaches on a shared text classification benchmark. Each model is evaluated under a consistent experimental protocol using the same dataset splits, evaluation metrics, and result logging infrastructure. The secondary goal is to quantify the tradeoff between model complexity, input representation quality, and predictive performance, with particular attention to how much improvement sequence-aware deep learning models provide over bag-of-words baselines.

3. Implementation Description

3.1 Repository and Codebase Overview

3.1.1 Repository Structure

The codebase is organized such that each model is in its own directory and utilities that are shared across several models are found in `utils/`:

```
CAP5610TermProject/
|-- Linear_SVM/
|-- Kernel_SVM/
|-- Naive_Bayes/
|-- Decision_Tree/
|-- Random_Forest/
|-- Logistic_Regression/
|-- LSTM/
|-- MultiLayer_Perceptron/
|-- CNN/
|-- Transformers/
|-- Large_Language_Model/
|-- utils/
|-- requirements.txt
`-- README.md
```

3.1.2 Shared Utilities

The `utils/` package exports a common API that all models scripts can utilize:

```
utils/
|-- args.py                    # common CLI flags
|-- data.py                    # HuggingFace yelp_review_full loader + split
```

```
|-- embeddings.py              # GloVe / fastText loaders for CNN
|-- evaluation.py              # metrics, plots, results logging
|-- randomness.py              # DEFAULT_SEED, set_random
|-- text_features.py           # shared TF-IDF vectorizer
|-- timer.py                   # timed_step context manager
`-- tuning.py                  # Optuna driver + best-config persistence
```

3.1.3 Model Specific Directories

Each model's directory contains its script, a tuning log, a results log, a saved best-config, and a confusion matrix for the best final result. For example, `CNN/`:

```
CNN/
|-- CNN.py                     # runnable model script
|-- confusion_matrix_cnn.png   # confusion matrix from best final run
|-- best_config.json           # winning hyperparameters from tuning
|-- results_log.md             # every runs metrics, params, runtime
`-- tuning_log.md              # every Optuna trials params + score
```

3.2 Data Preparation and Experimental Pipeline

3.2.1 Data Loading Workflow

All models read the dataset through a single entry point: `utils.data.load_yelp_data()`, which calls `datasets.load_dataset("yelp_review_full")`. The dataset downloads on first use and is cached locally. Following runs then read from cache.

3.2.2 Training/Validation/Test Protocol

The official split of the dataset is 650,000 training reviews and 50,000 test reviews. The loader then takes an additional validation split (by default 10%) out of the training partition using `sklearn.model_selection.train_test_split`. The test partition is never touched during tuning. Validation runs use a default 150,000-review subsample of training data for fast iteration; final runs use the full 650,000-review training set (unless specified otherwise for some models) and evaluate on the held-out 50,000-review test partition (triggered by `--final`).

3.2.3 Text Preprocessing

Sparse-feature models (Logistic Regression, Naive Bayes, Decision Tree, Random Forest, Linear/Kernel SVM, MLP) consume TF–IDF features built directly from raw review text by `TfidfVectorizer`, which lowercases, tokenizes, and applies `strip_accents="unicode"` internally. Sequence models (CNN, LSTM, Transformer) lowercase the text and replace punctuation with whitespace before tokenizing on whitespace, then map tokens to fixed-size index sequences (CNN: `max_len = 256`; LSTM: `max_len = 512`; Transformer: `max_len = 160`). Tokens outside of the vocabulary collapse to `<unk>` and zero-padded positions are masked at the embedding layer. The decoder-only LLM (default `distilgpt2`) bypasses this pipeline entirely and instead applies the model's pretrained HuggingFace `AutoTokenizer` (subword BPE) directly to the raw text, padding/truncating to `max_length = 256`.

3.2.4 Feature Engineering

TF–IDF: `fit_tfidf_features()` fits a single `TfidfVectorizer` with `sublinear_tf=True` (dampens the contribution of repeated words within a review via $1 + \log(\text{tf})$) and four tunable parameters: `max_features` (cap on the vocabulary size), `ngram_range` (whether to include bigrams in addition to unigrams), `min_df` (drop terms appearing in fewer than this many documents, removing rare/typo tokens), and `max_df` (drop terms appearing in more than this fraction of documents, removing extremely common words). Final feature dimensionalities range from 10,000 (MLP) to 40,000 (Decision Tree, Random Forest, Logistic Regression).

Word vocabulary: Each word-level sequence model builds its own top N vocabulary from the training text with index 0 reserved for the padding token (`<pad>`) and index 1 reserved for the unknown word token (`<unk>`). CNN: 25,000 words (fixed); LSTM: 30,000 words (tuned over 10k–75k); Transformer: 15,000 words (tuned over 15k–30k). Reviews are truncated or zero-padded to the model's `max_len`.

Embeddings: The CNN supports either a randomly initialized 256-dimensional embedding lookup (trained from scratch) or one of several pretrained sources: GloVe 6B 100d/300d, GloVe 42B 300d, GloVe 2024 WikiGigaword 300d, and fastText wiki-news subword 300d, which can all be selected with mutually exclusive CLI flags. Pretrained matrices are loaded into the embedding layer at model construction time. The LSTM uses a trainable `nn.Embedding` lookup tuned over 128, 256, 300 dimensions with no pretrained option. The Transformer learns both token and positional embeddings from scratch (token embedding tuned over 128, 256 dimensions, positional embedding sized to `max_len`). The decoder-only LLM gets its subword embeddings from the pretrained `distilgpt2` checkpoint and tunes them jointly with the rest of the model.

3.2.5 Randomness and Reproducibility

A single global seed (`DEFAULT_SEED = 0`) is set at the start of every run via `set_random_seed()`, which seeds all of the following: Python's `random`, NumPy, and Torch (including CUDA), and pins `torch.backends.cudnn.deterministic = True` with `benchmark = False`. The same seed is passed into the dataset splitter and into every Optuna study, so re-running any script reproduces the same train/validation split and the same trial order.

3.2.6 Hyperparameter Tuning Workflow

Tuning is invoked with the `--tune` flag and uses Optuna Bayesian optimization through `utils.tuning.tune_model()`. Each model defines an `objective()` that samples a configuration, builds features, trains on a subset of the training data ranging from roughly 6,000 reviews (LLM) to 650,000 reviews (CNN, Linear SVM) depending on the model's compute cost per trial, and returns macro F1 on the validation split. Trial budgets range from 6 (Logistic Regression) to 30 (CNN), with most models in the 15–20 range. When tuning completes, `save_best_config()` writes the winning configuration plus metadata (seed, embedding source, train size, validation split, trial count, best macro F1) to `best_config.json`, and the tuning helper appends a Markdown summary of all trials to `tuning_log.md`. Following runs load that JSON automatically unless `--default` is passed. Naive Bayes is the exception: `--tune` runs a manual alpha sweep over a fixed grid rather than an Optuna study, and writes `nb_alpha_sweep.png` instead of a tuning log.

3.3 Traditional Machine Learning Models

3.3.1 Logistic Regression

Folder/Script: `Logistic_Regression/Logistic_Regression.py`
Team Member: Mohammed Mamdouh

Model Description:
A multinomial logistic regression classifier from `sklearn.linear_model.LogisticRegression`. The model learns one linear score per class over the TF–IDF feature vector and applies a SoftMax based cross entropy objective for review rating prediction. Unlike Naive Bayes, it does not assume feature independence, but it is still a linear bag of words model and therefore only captures limited local order through the chosen n-gram features.

Input Representation: Sparse TF–IDF features built directly in the script with `TfidfVectorizer`. The vectorizer uses `sublinear_tf=True`, `strip_accents="unicode"`, and `dtype=np.float32`. The tuned configuration keeps 40,000 features, includes unigrams and bigrams, and sets `min_df = 2`, `max_df = 0.9`.

Training Procedure: Validation runs load a 150,000 review subset and hold out 10% for validation where final runs fit on the full 650,000-review training split and evaluate on the 50,000 review test split. Optimization is handled by scikit-learn rather than a manual training loop. The script uses `lbfgs` for `l2` regularization and switches to `saga` when `elasticnet` is selected.

Hyperparameters: The tunable parameters and their Optuna search spaces are:

Parameter	Type	Search Space
`tfidf_features`	Categorical	{20000, 40000, 60000}
`ngram_max`	Integer	{1, 2}
`min_df`	Categorical	{1, 2, 5}
`max_df`	Categorical	{0.9, 0.95, 1.0}
`C`	Float (log)	$[0.25, 8.0]$
`penalty`	Categorical	{`l2`, `elasticnet`}
`max_iter`	Categorical	{200, 300, 500}
`l1_ratio`	Float	$[0.05, 0.95]$ when `elasticnet` is used

The convergence tolerance remains fixed at `1e-4`.

Tuning Strategy: Optuna Bayesian optimization with 6 trials on a 100,000 review training subset and a 10% validation split. Search space: `tfidf_features` $\in \{20000, 40000, 60000\}$, `ngram_max` $\in \{1, 2\}$, `min_df` $\in \{1, 2, 5\}$, `max_df` $\in \{0.9, 0.95, 1.0\}$, `C` $\in [0.25, 8.0]$ on a log scale, `penalty` $\in \{$`l2`, `elasticnet`$\}$, `max_iter` $\in \{200, 300, 500\}$, and `l1_ratio` $\in [0.05, 0.95]$ when elastic net is active. The saved best configuration uses `l2` regularization with $C \approx 0.379$ and achieved validation macro F1 of 0.6002 during tuning.

Implementation Notes: The full pipeline stays lightweight: TF–IDF fitting, model training, evaluation, and artifact saving all happen in one script while reusing the shared repo utilities for dataset loading, metrics, and logging. Because the representation remains sparse end-to-end and training is CPU-only, Logistic Regression serves as one of the fastest strong baselines in the project. Its final macro precision, macro recall, and macro F1 are all tightly clustered near 0.615, which suggests the model is already making fairly balanced decisions across classes and that most of the remaining limitations comes from the linear TF–IDF representation from being a non deep learning model rather than an easily fixable training issue.

3.3.2 Naive Bayes

Folder/Script: `Naive_Bayes/naive_bayes.py`
Team Member: Benedetto Falin

Model Description: Complement Naive Bayes (CNB) from `sklearn.naive_bayes.ComplementNB`. CNB is a variant of Multinomial Naive Bayes that estimates class-conditional probabilities using the *complement* of each class rather than the class itself. For each class c, it computes the likelihood of a feature given all other classes combined, which has been shown to produce better-calibrated weight estimates on imbalanced or heavily overlapping text corpora than standard Multinomial NB. The model assumes conditional independence of features given the class label (the Naive Bayes assumption), meaning word co-occurrence and order are not modeled.

Input Representation: TF-IDF vectors produced by the shared `fit_tfidf_features()` helper with `sublinear_tf=True`, `max_features = 20,000`, `ngram_range = (1, 2)`, `min_df = 5`, `max_df = 0.9`. Bigrams are included so that two-word phrases such as "not good" or "really great" are represented as single features, partially compensating for the model's inability to reason about word order at inference time. The resulting sparse matrix is passed directly to the sklearn fit and predict methods with no further transformation.

Training Procedure: CNB training is a single-pass frequency count followed by a log-probability computation – there is no iterative optimization. Final runs fit on the full 585,000-review training partition and evaluate on the 50,000-review test partition. Validation runs use the 150,000-review subsample with a 10% validation split for the alpha sweep. Training is CPU-only and completes in under 30 seconds on the full partition.

Hyperparameters: The only tunable model hyperparameter is `alpha`, the Laplace/Lidstone additive smoothing coefficient. Higher alpha values add more probability mass to unseen feature-class combinations, preventing zero-probability estimates that would dominate the log-likelihood. The four TF-IDF parameters listed above are fixed rather than tuned for this model.

Tuning Strategy: Rather than Optuna, a manual grid sweep over a fixed list of alpha values is performed when `--tune` is passed:

$$\alpha \in \{0.01,\ 0.05,\ 0.1,\ 0.3,\ 0.5,\ 1.0,\ 2.0,\ 5.0,\ 10.0,\ 15.0,\ 20.0,\ 25.0,\ 30.0,\ 35.0,\ 40.0,\ 45.0\ 50.0\}$$

For each candidate, the model is fit on the training subset and validation accuracy is recorded. The alpha achieving the highest validation accuracy is saved to `nb_best_config.json` and used for all subsequent runs. A validation accuracy versus alpha plot is saved as `nb_alpha_sweep.png`. The best alpha found was

45. This value is higher than what was observed under the original unigram-only pipeline (where alpha of 30 was optimal), and the increase is attributable to the addition of bigrams in the TF-IDF feature set. Bigram features are considerably more sparse than unigram where a phrase like "not great" appears far less frequently than either constituent word, which reduces the average discriminative reliability of individual features and requires heavier smoothing to prevent the model from over-fitting to noisy frequency counts.

Implementation Notes: The unusually high optimal alpha is an interpretable finding in itself. Standard smoothing values (0.1 to 1.0) are sufficient when classes are well-separated in feature space. The fact that alpha must reach 45 before validation accuracy peaks indicates that individual word frequencies are weak discriminators across the five classes and heavy smoothing is needed to prevent the model from over-committing to any single term. This is consistent with the known difficulty of the middle star ratings (2, 3, and 4 stars), where reviews share substantial vocabulary regardless of the assigned rating.

3.3.3 Decision Tree

Folder/Script: `Decision_Tree/Decision_Tree.py`
Team Member: Keating Sane

Model Description: A single decision tree classifier from `sklearn.tree.DecisionTreeClassifier`. The tree is built top-down by repeatedly choosing one TF–IDF feature and a threshold that best separates the five star-rating classes (using Gini impurity), then recursing on each side. Predictions follow the chain of feature-vs-threshold tests from the root to a leaf, where the majority class is returned. Implemented as a baseline against the bagged ensemble (Random Forest).

Input Representation: TF–IDF vectors produced by the shared `fit_tfidf_features()` helper with `sublinear_tf=True`, `max_features = 40,000`, `ngram_range = (1, 2)`, `min_df = 10`, `max_df = 0.9`. No additional preprocessing.

Training Procedure: Final runs fit a single tree on the full 650,000-review training partition with the tuned config and evaluate on the 50,000-review test partition. Validation runs use a 150,000-review training subsample with a 10% validation split for fast iteration. CPU-only.

Hyperparameters: The tunable parameters are `max_depth` (maximum tree depth, which caps model capacity and the rate at which the tree can carve up the feature space), `min_samples_leaf` (minimum training rows per leaf, which prevents the tree from creating tiny leaves that memorize individual reviews), and the four TF–IDF parameters listed above. The splitting criterion is sklearn's default Gini impurity.

Tuning Strategy: Optuna Bayesian optimization with 20 trials on a 200,000-review tuning subset (10% validation split). Search space: `tfidf_features` $\in$ {5k, 10k, 20k, 40k}, `ngram_max` $\in$ {1, 2}, `min_df` $\in$ {3, 5, 10}, `max_df` $\in$ {0.85, 0.9, 0.95}, `max_depth` $\in$ {50, 100, 150, 300, None}, `min_samples_leaf` $\in [1, 10]$. macro F1 on the validation split was the objective.

Implementation Notes: Tree fitting on the 200k tuning subset takes well under a minute per trial on CPU, but the full 650k-review final fit grows to roughly an hour as the tree depth scales. With the best tuned settings, the model memorizes most of the training set: train accuracy is far above the macro F1 it achieves on validation/test. The model was kept as an interpretable baseline to compare with the ensemble.

3.3.4 Random Forest

Folder/Script: `Random_Forest/Random_Forest.py`
Team Member: Keating Sane

Model Description: Ensemble of decision trees from `sklearn.ensemble.RandomForestClassifier`. Each tree is fit on a bootstrap sample of the training rows with a random subset of features considered at every split (`max_features = "sqrt"`); class predictions are averaged across the forest. The randomized feature subsampling reduces the variance that the single Decision Tree exhibits.

Input Representation: TF–IDF vectors from `fit_tfidf_features()` with `sublinear_tf=True`, `max_features = 40,000`, `ngram_range = (1, 2)`, `min_df = 10`, `max_df = 0.85`. Identical pipeline to the single tree, with a slightly tighter `max_df`.

Training Procedure: Final runs fit a 500-tree forest on the full 650,000-review training partition with the tuned config and evaluate on the 50,000-review test partition. Validation runs use a 150,000-review training subsample with a 10% validation split. Tree fitting is parallelized across cores via `n_jobs = -1`; CPU-only.

Hyperparameters: The tunable parameters are mirrored from the Decision Tree, with two additions: `n_estimators` (number of trees in the forest) and `max_features` (number of features randomly considered at each split).

Tuning Strategy: Optuna Bayesian optimization with 20 trials on a 200,000-review tuning subset (10% validation split). Search space mirrors the Decision Tree's parameters and adds `n_estimators` $\in$ {100, 200, 300, 500} and `max_features` $\in$ {"sqrt", "log2"}. macro F1 on the validation split was the objective.

Implementation Notes: A 500-tree forest on the 200k tuning subset fits in roughly three minutes per trial with `n_jobs = -1`; the full 650k-review final fit grows to about 44 minutes. Bootstrap sampling plus the `max_features = "sqrt"` per-split feature subset force each tree to see a different slice of the data and consider a different subset of words at every split, so the trees make different mistakes and the variance that the single Decision Tree exhibits averages out.

3.3.5 Linear SVM

Folder/Script: `Linear_SVM/Linear_SVM.py`
Team Member: Alexander Nardi

Model Description: A Linear Support Vector Machine (Linear SVM) from `sklearn.svm.LinearSVC`. This model learns a set of linear decision boundaries that maximize the margin between the review classes (1-5 stars). Linear SVM's operate directly in the original, high dimensional feature space (in this case, TF-IDF's), which make it extremely efficient for sparse, high-dimensional data. The model optimizes hinge loss with L2 regularization.

Input Representation: TF–IDF vectors produced by the `sklearn.feature_extraction.text.TfidfVectori` with optimal maximum features found in tuning and english stop-words. The standard TF–IDF's sparse representation is well-suited for Linear SVMs, seeing as the model scales linearly, as the name implies, with the amount of non-zero features.

Training Procedure: The model is trained using a coordinate descent, implemented in sklearn. Final runs fit on the full training partition and evaluate on the test set. Validation runs for tuning use a subsampled training set with a 10% validation split for hyperparameter tuning. Training is CPU-only and completes relatively quickly in only about 100 seconds or so.

Hyperparameters: The primary hyperparameter is `C`, the inverse regularization strength. Smaller values of `C` enforce stronger regularization, resulting in a wider margin but resulting in potentially higher bias. Whereas larger values do the opposite, weakening regularization, and allowing the model to fit the training data more closely. The secondary hyperparameter was the maximum length `max_features`, which is the number of features considered

Tuning Strategy: Optuna Bayesian optimization with 15 trials on the maximum 650,000-review training set, with a 10% validation split. The full training set is allowed as training is relatively quick, and this allows us to find optimal hyperparameters for the ones listed above. `C` is selected as the best float between {0.1, 10.0}, whereas `max_features` is selected between {5000, 10,000, & 15,000}. The optimal hyperparameters is noted to be `C = 10619348907224617` and `max_features = 15,000`. Maximizing the macro F1 score was used as the objective for hyperparameter tuning.

Implementation Notes: Linear SVM was actually particularly well-suited for this task due to the high dimensionality and the inherent sparsity in a TF–IDF Bag-of-Words approach. Even then, the model performed rather shockingly high on a NLP task, getting around 57% accuracy and surpassing models which should theoretically have performed better as Linear SVM cannot process word order.

3.3.6 Kernel SVM

Folder/Script: `Kernel_SVM/Kernel_SVM.py`
Team Member: Anthony Mahon

Model Description: A kernel Support Vector Machine using `sklearn.svm.SVC` with a Radial Basis Function (RBF) kernel. SVMs learn a decision boundary that maximizes the margin between the nearest training points (support vectors) of each class. The soft-margin objective allows some misclassifications, controlled by the penalty parameter C:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_i \xi_i \quad \text{s.t.} \quad y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1 - \xi_i,\ \xi_i \geq 0.$$

The kernel trick replaces the explicit mapping ϕ with a kernel function evaluated on pairs of input vectors, enabling the model to find non-linear boundaries without ever computing the high-dimensional feature map. The RBF kernel measures similarity between two TF–IDF vectors $\mathbf{x}$ and $\mathbf{x}'$ as:

$$K(\mathbf{x}, \mathbf{x}') = \exp\left(-\gamma\|\mathbf{x} - \mathbf{x}'\|^2\right),$$

where γ (left at the sklearn default, $1/(\text{n_features} \cdot \text{Var}(\mathbf{X}))$) controls how quickly similarity decays with distance. The multiclass prediction uses one-vs-one voting across all $\binom{5}{2} = 10$ binary sub-classifiers.

Input Representation: TF–IDF vectors produced by `sklearn.feature_extraction.text.TfidfVectorize` with `stop_words='english'` and a vocabulary capped at `max_features`. TF–IDF assigns higher weight to words that are frequent within a specific review but rare across the full corpus (e.g., "undercooked" or "phenomenal"), making it a strong signal for sentiment-loaded vocabulary. No additional normalization is applied beyond what `TfidfVectorizer` performs internally (`norm='l2'` by default).

Training Procedure: Exact kernel SVM training requires computing and storing the full $n \times n$ kernel matrix, giving $O(n^2)$ memory and $O(n^3)$ time complexity. Training on the full 650,000-review corpus is therefore infeasible. The model is trained on a stratified subsample of 10,000 reviews by default (up to 20,000 with `--size N`). A 20% stratified validation split is carved from this subsample for tuning; final runs evaluate on the official 50,000-review test partition. Training is CPU-only.

Hyperparameters: Two parameters were tuned:

- `C`: soft-margin penalty. Smaller values allow more misclassifications but produce wider, more generalizable margins; larger values force a tighter fit to the training data.
- `max_features`: vocabulary cap for the TF–IDF vectorizer. Controls the size of the input space presented to the kernel.

Tuning Strategy: Optuna Bayesian optimization with 15 trials on a 10,000-sample subsample (20% validation split). Search space: $C \in [0.1, 10.0]$ (log scale), `max_features` $\in \{5000, 10000, 15000\}$. Macro-F1 on the validation split was the objective. Notably, the best Optuna configuration did not outperform a manual setting of $C = 1$ (the sklearn default), which was ultimately selected. This is expected behavior: with only 10,000 training samples the validation signal is noisy, and the well-validated default $C = 1$ generalizes more reliably than configurations tuned on a small subsample. Best config: $C = 1$, `max_features` = 10,000.

Implementation Notes: The training-set size cap is the dominant limitation of this model. The linear SVM and logistic regression train on 65 times more data and comfortably outperform the kernel SVM as a consequence. The RBF kernel's theoretical advantage (non-linear boundaries) cannot compensate for this data deficit. Theoretically, the model should be among the top performers, but in practice, it is difficult to use its structure within reasonable computation times.

3.4 Deep Learning Models

3.4.1 Multi-Layer Perceptron

Folder/Script: `MultiLayer_Perceptron/ml_perceptron.py`
Team Member: Benedetto Falin

Model Description: A fully connected feedforward neural network implemented in PyTorch (`torch.nn.Sequential`). The MLP takes a fixed-size TF-IDF vector as input and learns a non-linear mapping to five class logits via two hidden layers with batch normalization, ReLU activations, and dropout regularization. Unlike Naive Bayes, the MLP can learn interactions between input features, but like all TF-IDF-based models it has no access to word order or sequential context. Trained end-to-end with cross-entropy loss via backpropagation.

Input Representation: TF-IDF vectors from the shared `fit_tfidf_features()` helper with `sublinear_tf=True`, `max_features = 10,000`, `ngram_range = (1, 2)`, `min_df = 5`, `max_df = 0.9`. The sparse matrix is converted to a dense `float32` PyTorch tensor in chunks of 10,000 rows to avoid attempting a full dense allocation of approximately 20 GB. The resulting dense tensor is wrapped in a `TensorDataset` and served through a `DataLoader` with `pin_memory=True` for efficient GPU transfer.

Architecture:

$$\mathbb{R}^{10000} \xrightarrow{\text{Linear}} \mathbb{R}^{h_1} \xrightarrow{\text{BN, ReLU, Dropout}} \mathbb{R}^{h_2} \xrightarrow{\text{BN, ReLU, Dropout}} \mathbb{R}^{5}$$

where h_1 and h_2 are tunable hidden layer widths. Batch normalization is applied before each ReLU to stabilize training and reduce sensitivity to the learning rate. No softmax is applied at the output; `nn.CrossEntropyLoss` applies the log-softmax internally. The default architecture uses $h_1 = 1024$, $h_2 = 512$.

Training Procedure: Optimizer is Adam with a `ReduceLROnPlateau` scheduler that halves the learning rate when validation accuracy does not improve for 2 consecutive epochs. Early stopping with patience of 3 halts training when validation accuracy has not improved for 3 consecutive epochs, and the best-validation-accuracy checkpoint is restored before evaluation. Final runs train on the full 585,000-review training partition for up to 15 epochs and evaluate on the 50,000-review test partition. Validation runs use the 150,000-review subsample.

Hyperparameters: The tunable parameters and their Optuna search spaces are:

Parameter	Type	Search Space
`hidden1`	Categorical	{512, 1024, 2048}
`hidden2`	Categorical	{256, 512}
`dropout`	Float	$[0.1, 0.6]$ in steps of 0.1
`lr`	Float (log)	$[10^{-4},\ 5 \times 10^{-3}]$
`weight_decay`	Float (log)	$[10^{-5},\ 10^{-2}]$

Tuning Strategy: Optuna Bayesian optimization with 20 trials via `utils.tuning.tune_model()`. Each trial trains for up to 5 epochs with patience of 2 on the 150,000-review validation subset and returns the best validation accuracy achieved. The TPE sampler biases subsequent trials toward configurations that performed well in earlier trials. The best configuration found was $h_1 = 512$, $h_2 = 512$, `dropout` $= 0.6$, `lr` $= 0.0001$, `weight_decay` $= 5 \times 10^{-5}$, achieving a validation accuracy of 62.30% versus the default baseline of 60.28%.

Implementation Notes Early runs without early stopping exhibited severe overfitting: training accuracy climbed above 85% while validation accuracy plateaued near 61%, indicating the model's capacity far exceeds what the TF-IDF representation can supply in useful signal. Batch normalization and increased dropout (0.6 in the best config) were the two changes that most reduced the train/val gap. The performance ceiling near 61% is consistent with published results for bag-of-words MLP approaches on Yelp Full and is attributed primarily to the loss of word order at the input stage rather than insufficient model capacity.

3.4.2 Long Short-Term Memory (LSTM)

Folder/Script: `LSTM/LSTM.py`
Team Member: Alexander Nardi

Model Description: A bidirectional Long Short-Term Memory (LSTM) network implemented in PyTorch. The model processes each review as a ordered sequence of token indices rather than a TF–IDF vector via constructing an indexed vocabulary. This allows it to preserve the word order and sequential

context of the review. The tokens are then mapped to a learned embedding layer and passed through the bidirectional LSTM network. The final forward and backward hidden states are concatenated and passed through a dropout layer, and finally projected onto the 5 class logits with a fully connected (FC) layer. The model is trained with Cross-Entropy (CE) loss.

Input Representation: Reviews are lowercased and then punctuation is replaced with whitespace for text cleaning. The text is then tokenized using a whitespace delimiter. A vocabulary is built from the most frequent words of the reviews, with index 0 reserved for `pad` and 1 reserved for `unk`. `Pad` is used when you need to pad a review to the `max_len`, whereas `unk` is reserved for unknown words (i.e greatttt). The default configuration uses `max_vocab = 50000` and `max_len = 256`, while tuning evaluates vocabulary sizes from 10,000 to 75,000 and sequence lengths of 128, 256, 512, and 1024.

Architecture:

$$\text{Token indices} \rightarrow \text{Embedding}(V, E) \rightarrow \text{BiLSTM} \rightarrow [h_{\text{forward}}; h_{\text{backward}}] \rightarrow \text{Dropout} \rightarrow \mathbb{R}^5$$

The LSTM uses `batch_first=True`. For the bidirectional model, the final hidden state from the forward direction and the final hidden state from the backward direction are concatenated before classification. No softmax is applied at the output because `nn.CrossEntropyLoss` applies log-softmax internally. Optimizer used is AdamW.

Training Procedure: The model is trained with AdamW using a fixed `weight_decay = 1e-4`. Gradients are clipped to a maximum L2 norm of 1.0 to stabilize recurrent training. Validation runs use a 150,000-review subsample with a 10% validation split, while final runs train on the full 650,000-review training partition and evaluate on the 50,000-review test partition. Early stopping monitors validation loss with a patience of 3 epochs, and the best validation-loss checkpoint is restored before evaluation.

Hyperparameters: The tunable parameters and their Optuna search spaces are:

Parameter	Type	Search Space
`max_vocab`	Categorical	{10000, 20000, 30000, 40000, 50000, 75000}
`max_len`	Categorical	{128, 256, 512}
`embed_dim`	Categorical	{128, 256, 300}
`hidden_dim`	Categorical	{128, 256}
`num_layers`	Categorical	{1, 2}
`bidirectional`	Categorical	{`True`}
`dropout`	Float	$[0.2, 0.5]$
`lr`	Float (log)	$[10^{-4}, 10^{-3}]$
`batch_size`	Categorical	{64, 128}
`epochs`	Integer	$[3, 15]$

Tuning Strategy: Optuna Bayesian optimization is run for 15 trials on a 50,000-review tuning subset with a 10% validation split. Each trial builds a vocabulary, tokenizes the train and validation sets using the `max_vocab` and `max_len`, trains the LSTM with early stopping with a patience of 3. Maximized Macro F1 score is the training objective. The best epoch is recorded into `best_config.json`'s metadata so following runs can compare.

Best Configuration: The best configuration found via Optuna was:

Parameter	Value
`max_vocab`	30,000
`max_len`	1024
`embed_dim`	300
`hidden_dim`	256
`num_layers`	2
`bidirectional`	True
`dropout`	0.277
`lr`	5.44×10^{-4}
`batch_size`	64
`epochs`	4

This configuration achieved a validation macro-F1 of **0.5974**.

Implementation Notes: Training is done via GPU (its recommended to have a strong NVIDIA GPU for CUDA capabilities). Final fit was is completed in around 90 seconds on a 4080 SUPER. It should be noted that the optimization found a very computation hungry LSTM to be most optimal. In fact, it selected for all of the most computationally complex options.

3.4.3 Convolutional Neural Network

Folder/Script: `CNN/CNN.py`
Team Member: Keating Sane

Model Description: A Kim (2014) style TextCNN implemented in PyTorch. An embedding layer maps token indices to dense vectors, several 1D convolutional filters of different kernel widths run in parallel over the embedded sequence (acting as learned n-gram detectors), max-over-time pooling collapses each feature map to a single value per filter, the pooled outputs are concatenated, passed through dropout, and projected to five class logits via a fully connected layer. Trained end-to-end with cross-entropy loss.

Input Representation: Reviews are lowercased, punctuation is replaced with whitespace, and the text is tokenized on whitespace. The training corpus produces a 25,000-word vocabulary with index 0 reserved for the padding token (`<pad>`) and index 1 reserved for the unknown-word token (`<unk>`). Each review is mapped to a 256-token index sequence (truncated or zero-padded). The embedding source is chosen at the CLI: either a randomly initialized 256-dimensional lookup (default) or one of five pretrained options: GloVe 6B 100d, GloVe 6B 300d, GloVe 42B 300d, GloVe 2024 WikiGigaword 300d, or fastText wiki-news subword 300d. Pretrained matrices are loaded directly into the embedding layer at model construction.

Architecture: Embedding ($V \times E$) → parallel `Conv1d` filters at multiple kernel widths → ReLU → max-over-time pooling per filter → concatenate pooled outputs → dropout → fully connected layer producing 5-class logits. Padding positions are masked at the embedding via `padding_idx = 0`. Optimizer is Adam; gradients are clipped to a max L2 norm of 1.0.

Training Procedure: Final runs train on the full 650,000-review training partition and evaluate on the 50,000-review test partition. Validation runs use a 150,000-review training subsample with a 10%

validation split. Each epoch ends with a held-out validation pass that tracks the lowest-loss checkpoint; training stops early if validation loss has not improved for 3 consecutive epochs, and the lowest-loss checkpoint is restored before evaluation. Auto-selects MPS, CUDA, or CPU at startup.

Hyperparameters: The tunable parameters are `num_filters` (number of feature maps per kernel width), `filter_sizes` (the set of kernel widths run in parallel, wider filters capture longer n-grams), `dropout` (regularization on the concatenated feature vector before the fully connected layer), `lr` (Adam learning rate), `batch_size`, and `epochs` (effective via early stopping). The embedding source is selected at the CLI rather than tuned; the choice is recorded in `best_config.json`'s metadata so following runs can detect a mismatch.

Tuning Strategy: Optuna Bayesian optimization with 30 trials on the full 650,000-review training partition (10% validation split). Search space: `num_filters` $\in \{100, 200, 300\}$, `filter_sizes` $\in \{$"3,4,5", "2,3,4,5"$\}$, `dropout` $\in [0.2, 0.6]$, `lr` $\in [10^{-4}, 10^{-2}]$ (log scale), `batch_size` $\in \{64, 128\}$, `epochs` $\in [5, 15]$. macro F1 on the validation split was the objective. The best epoch identified by early stopping is written back into `best_config.json` so final runs reproduce that training duration.

Implementation Notes: Training is GPU-bound; on an RTX 5090 a final fit on 650k reviews completes in under a minute per epoch. Because the embedding source materially affects training, the saved choice is recorded in `best_config.json`'s metadata so following runs use the same embeddings the config was tuned with.

3.4.4 Transformer (Encoder Only)

Folder/Script: `Transformers/Transformers.py`
Team Member: Anthony Mahon

Model Description: An encoder-only Transformer trained entirely from scratch on the YelpReview-Full dataset, implemented in PyTorch. The model follows the standard Transformer encoder design introduced by Vaswani et al. (2017): stacked multi-head self-attention blocks allow every token in a review to attend to every other token simultaneously, capturing long-range dependencies that recurrent models process sequentially. Unlike BERT or other pre-trained models, no external linguistic knowledge is used; every parameter, including the word embeddings, is initialized randomly and learned solely from the training reviews. This means that the model is not only learning the structure of the reviews, but also the structure of the English language itself.

Input Representation: Raw review text is lowercased and stripped of punctuation via a regular expression filter (`r"[^\w\s]"`). The resulting token stream is split on whitespace and mapped to integer IDs using a frequency-ranked vocabulary built from the training corpus, capped at `max_vocab=30,000` entries. Words outside the top 30k most common collapse to `<unk>` (index 1); sequences are truncated to a fixed length `max_len`, producing an integer matrix of shape [`batch_size` $\times$ `max_len`].

Architecture: The model is implemented as `EncoderClassifier` in `Transformers/Transformers.py`, and consists of four components:

1. **Token embedding** (`nn.Embedding`): maps each token ID to a dense `embed_dim`-dimensional vector. Padding positions use `padding_idx=0` and are excluded from attention via a key-padding mask passed to the encoder.

2. **Positional embedding** (`nn.Embedding`): a second learnable embedding maps each position index $(0, 1, \ldots, \texttt{max_len} - 1)$ to an `embed_dim`-dimensional vector. Token and position embeddings are summed element-wise, giving the model awareness of word order.
3. **Transformer encoder stack** (`nn.TransformerEncoder`): `num_layers` `TransformerEncoderLayer` blocks, each containing multi-head self-attention followed by a position-wise feed-forward sub-network with GELU activation. Self-attention is computed as:

$$\mathrm{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}, \quad d_k = \frac{\texttt{embed_dim}}{\texttt{num_heads}}.$$

   The `batch_first=True` convention is used throughout.
4. **Classification head**: non-padding token representations are mean-pooled (masking padding positions to prevent them from influencing the average), passed through layer normalization and dropout, and projected to five logits by a linear layer. The highest logit selects the predicted star rating.

Training Procedure: The optimizer used is AdamW (`torch.optim.AdamW`) with cross-entropy loss over the five-class softmax output. Training runs for `epochs` full passes over the training set in mini-batches of size `batch_size`. After each epoch, macro-F1 is computed on the validation split and the best-scoring model state is saved separately; the best state is restored before final evaluation. Final runs use the full 650,000-review training partition (minus a 5% stratified validation slice to identify the best model state) and require a GPU; the final reported run used a Tesla T4 provided through Google Collab.

Hyperparameters: Ten Optuna trials were run on a 20,000-sample training subsample (10% validation split). The search space was:

Parameter	Type	Search Space
`max_vocab`	Categorical	{15000, 25000, 30000}
`max_len`	Categorical	{96, 128, 160}
`embed_dim`	Categorical	{128, 256}
`num_heads`	Categorical	{4, 8}
`num_layers`	Categorical	{2, 3}
`ff_dim`	Categorical	{256, 512}
`dropout`	Float	[0.1, 0.3]
`epochs`	Categorical	{2, 3}
`batch_size`	Categorical	{32, 64}
`lr`	Float (log)	$[10^{-3},\ 7 \times 10^{-3}]$
`weight_decay`	Float (log)	$[10^{-5},\ 10^{-3}]$

The best trial achieved a validation macro-F1 of 0.4994. Best config: `max_vocab` = 15,000, `max_len` = 160, `embed_dim` = 256, `num_heads` = 8, `num_layers` = 2, `ff_dim` = 256, `dropout` = 0.2658, `lr` = 6.50×10^{-4}, `weight_decay` = 3.14×10^{-5}, `batch_size` = 64, `epochs` = 3.

Tuning Strategy: Optuna Bayesian optimization via `utils.tuning.tune_model()`, objective is macro-F1 on the validation split. The tuning subsample (20,000 reviews) was chosen as the largest size that balances computation time on a T4, allowing 10 meaningful trials within a reasonable time budget.

Implementation Notes: Training is GPU-bound; a full final run on 650k reviews with 3 epochs takes approximately 21 minutes on a Tesla T4. CPU users are warned at startup that training will be prohibitively slow. The model uses `enable_nested_tensor=False` in `nn.TransformerEncoder` for compatibility across PyTorch versions. The best hyperparameter configuration found by Optuna provided an increase across all metrics of 1%, at the cost of a 17-minute increase in computation time due to the new size of the embedding dimension. It should be noted that the improvement is not worth the expense taken, showcasing that the initial baseline, default parameters balanced performance with efficiency very well.

3.4.5 Large Language Model (Decoder Only)

Folder/Script: `Large_Language_Model/Large_Language_Model.py`
Team Member: Mohammed Mamdouh

Model Description A decoder only transformer finetuned for 5 class review classification through HuggingFace's `AutoModelForSequenceClassification`. Although the backbone model is a standard language model, this implementation uses it as a classifier by attaching a sequence level classification head and optimizing cross entropy loss on our dataset.

Pretrained Backbone: Our base model is `distilgpt2`. At load time, the script rewrites the configuration for `num_labels = 5` and populates label name mappings so predictions align with the repository's shared evaluation utilities. If the tokenizer does not define a padding token, the end of sequence token is reused for padding.

Tokenization and Input Format: Raw review text is passed directly to the pretrained HuggingFace tokenizer, which performs subword BPE tokenization. Inputs are truncated to a tunable `max_length` (128, 192, or 256 during tuning) and stored as `input_ids` plus `attention_mask`. Padding is applied dynamically per batch through `DataCollatorWithPadding`, which keeps memory use lower than padding the full dataset to a fixed length in advance.

Training Procedure: Non final runs use a 20,000 review subset with a 10% validation split. Final runs load the full 650,000-review training split, reserve 5% of it as an internal validation set for early stopping, then evaluate the best checkpoint on the 50,000-review test split. Optimization uses AdamW with a linear warmup schedule, gradient clipping at 1.0, mixed precision on CUDA, and early stopping off of the validation macro F1. After training, the best validation checkpoint is restored before the final evaluation pass.

Hyperparameters: The tunable parameters and their Optuna search spaces are:

Parameter	Type	Search Space
`max_length`	Categorical	{128, 192, 256}
`epochs`	Categorical	{1, 2}
`batch_size`	Categorical	{2, 4}
`eval_batch_size`	Categorical	{4, 8}
`learning_rate`	Float (log)	$[10^{-5}, 8 \times 10^{-5}]$
`weight_decay`	Float (log)	$[10^{-4}, 5 \times 10^{-2}]$
`warmup_ratio`	Float	$[0, 0.12]$
`grad_accum_steps`	Categorical	{1, 2, 4}

Patience is fixed at 2 in regular runs and reduced to 1 during tuning so poor trials end quickly.

Tuning Strategy: Optuna optimization with 8 trials on a 6,000 review subset and a 10% validation split. The backbone itself is fixed during each study where tuning only searches the fine-tuning configuration. Search space: `max_length` $\in \{128, 192, 256\}$, `epochs` $\in \{1, 2\}$, `batch_size` $\in \{2, 4\}$, `eval_batch_size` $\in \{4, 8\}$, `learning_rate` $\in [10^{-5}, 8 \times 10^{-5}]$, `weight_decay` $\in [10^{-4}, 5 \times 10^{-2}]$, `warmup_ratio` $\in [0, 0.12]$, and `grad_accum_steps` $\in \{1, 2, 4\}$. The saved best configuration for `distilgpt2` uses `max_length = 256`, `epochs = 2`, `batch_size = 2`, `eval_batch_size = 8`, learning rate $\approx 1.28 \times 10^{-5}$, weight decay ≈ 0.0053, and warmup ratio ≈ 0.0172, reaching macro F1 of 0.5926 during tuning.

Implementation Notes: The implementation pretokenizes the dataset into a small custom `Dataset` object and uses `DataCollatorWithPadding` so each batch is padded only to its local maximum sequence length. On CUDA, training uses automatic mixed precision through `torch.autocast` and `GradScaler` to reduce memory pressure. The best model state is tracked in memory using validation macro F1, restored after training, and also written to `Large_Language_Model/saved_model` so the exact fine-tuned checkpoint can be reused later.

3.5 Cross-Model Engineering Choices

3.5.1 Common CLI Design

Every model script imports `common_parser()` from `utils.args`, which registers four shared flags: `--tune` (run hyperparameter search), `--final` (full training set, evaluate on the test partition), `--discard` (run without writing any artifacts), and `--default` (ignore any saved `best_config.json` and use hard-coded defaults). Model-specific flags (e.g. the CNN's embedding-source selectors, the SVMs' `--size`, the LLM's checkpoint and resume flags) are layered on top of this shared parser. `--tune` is mutually exclusive with `--final` and `--default`.

3.5.2 Logging and Artifact Saving

Run output is structured by helpers in `utils.evaluation`: `print_run_header()` prints the model name and contextual info; `print_value_section()` prints the parameter table; `timed_step()` prints elapsed time for each step during loading/training. After evaluation, `save_results()` appends a Markdown section to `results_log.md` with accuracy, macro precision, macro recall, macro F1, runtime, parameters, and metadata; the same helper guards against overwriting a better run by comparing macro F1 against

the existing entry. For final runs, `plot_confusion_matrix()` writes a PNG figure into the model folder. `--discard` bypasses all writes.

3.5.3 Hardware/Runtime Considerations

Sparse feature models run on CPU and parallelize through `n_jobs=-1` where supported. The CNN, LSTM, Transformer, MLP, and LLM auto-select MPS, CUDA, or CPU at startup and use a fixed size token batch loader. Final run hardware varied: NVIDIA RTX 5090 (Linear SVM, CNN), AMD Ryzen 7 9800X3D (Decision Tree, Random Forest), AMD Ryzen 7 7800X3D (Naive Bayes, MLP), Tesla T4 (Transformer), and Intel i7-1165G7 (Kernel SVM). Final runtimes are recorded per-run in each `results_log.md`.

4. Usage Guide

This section explains how a user can setup our project to reproduce the results in this report.

4.1 Environment Setup

4.1.1 Prerequisites

Python 3.11 or newer, `pip`, and roughly 5 GB of free disk space for the cached dataset and pretrained embedding files. A CUDA or MPS capable GPU is recommended for the CNN, Transformer, and LLM but is not required. All other models run on CPU.

4.1.2 Installation

```
# clone
git clone <repo-url> && cd CAP5610TermProject

# create env (Python 3.11)
python -m venv .venv && source .venv/bin/activate

# install deps
pip install -r requirements.txt
```

4.1.3 Dataset Access

The dataset is fetched on first run via `datasets.load_dataset("yelp_review_full")` from HuggingFace. No credentials or manual downloads are required; following runs read from the local HuggingFace cache. Pretrained GloVe and fastText files used by the CNN download on demand the first time a corresponding embedding flag is passed.

4.2 Common Run Modes

Flag	Purpose	Notes
`--tune`	Run the Optuna hyperparameter search and save the best config.	Trains on a model-specific subset and evaluates on the validation split.

`--final`	Train on the full 650,000-review training partition (unless otherwise specified for some models) and evaluate on the 50,000 review test partition.	Loads the saved `best_config.json` unless `--default` is passed. Writes a confusion matrix PNG.
`--discard`	Skip writing any artifacts (results log, tuning log, best config, confusion matrix).	Useful for debugging.
`--default`	Ignore saved `best_config.json` and use the hard-coded defaults in the script.	Useful when rerunning with a different embedding source than the one the saved config was tuned with (e.g. CNN), or when comparing tuned vs. untuned baselines.

4.3 How to Run Each Model

Every model is invoked with `python <Directory>/<Script>.py` plus the desired flag combination. A typical workflow is to run `--tune` once to populate `best_config.json`, then run `--final` to produce the test-set numbers reported in Section 6.

Inputs: raw review text loaded automatically from the HuggingFace cache; for the CNN, an optional pretrained embedding flag.

Outputs: a printed run header, parameter table, timings, evaluation metrics, and (for final runs) a confusion matrix.

Artifacts produced: an appended section in the models `results_log.md`; for tuning runs, an appended section in `tuning_log.md` and a refreshed `best_config.json`; for final runs, `confusion_matrix_*.png`.

```
# Hyperparameter search (one time per model)
python Logistic_Regression/Logistic_Regression.py --tune
python Naive_Bayes/naive_bayes.py --tune
python Decision_Tree/Decision_Tree.py --tune
python Random_Forest/Random_Forest.py --tune
python Linear_SVM/Linear_SVM.py --tune
python Kernel_SVM/Kernel_SVM.py --tune
python MultiLayer_Perceptron/ml_perceptron.py --tune
python LSTM/LSTM.py --tune
python CNN/CNN.py --tune --glove-2024-wikigiga
python Transformers/Transformers.py --tune
python Large_Language_Model/Large_Language_Model.py --tune

# Final test-set evaluation using the saved best config
python Logistic_Regression/Logistic_Regression.py --final
python Naive_Bayes/naive_bayes.py --final
python Decision_Tree/Decision_Tree.py --final
python Random_Forest/Random_Forest.py --final
python Linear_SVM/Linear_SVM.py --final
python Kernel_SVM/Kernel_SVM.py --final
python MultiLayer_Perceptron/ml_perceptron.py --final
python LSTM/LSTM.py --final
python CNN/CNN.py --final --glove-2024-wikigiga
python Transformers/Transformers.py --final
python Large_Language_Model/Large_Language_Model.py --final
```

```

# Debugging: validation-mode run without writing anything
python CNN/CNN.py --discard --default
```

Additional flags. A few models layer extra options on top of the four shared flags:

- **CNN** (mutually exclusive embedding source selectors): `--glove-6b-100d`, `--glove-6b-300d`, `--glove-42b`, `--glove-2024-wikigiga`, `--fasttext-wiki-subword`. Omitting all of them uses a randomly initialized 256-dimensional embedding lookup.
- **Linear SVM** and **Kernel SVM**: `--size N` overrides the training subset size (defaults: 650,000 for Linear SVM, 10,000 for Kernel SVM).
- **Large Language Model**: `--model-name NAME` (default `distilgpt2`) chooses the decoder-only base model; `--smoke` runs a tiny model on a tiny subset for sanity checks; `--train-size N` overrides the non-final training subset size; `--trials N` sets the number of Optuna trials in `--tune` mode (default 8); `--resume` resumes from `Large_Language_Model/saved_model` and `--resume-from PATH` resumes from an arbitrary checkpoint. `--tune` additionally blocks `--resume`/`--resume-from`, and `--final` blocks `--smoke`.

4.4 Expected Output Files

Type	Location	Purpose
Best configuration	`<Model>/best_config.json`	Hyperparameters and metadata from the most recent tuning run.
Tuning logs	`<Model>/tuning_log.md`	Markdown record of every Optuna trial, noting parameters, val macro F1, and the runtime. Naive Bayes is the exception: it runs a manual alpha sweep producing `nb_alpha_sweep.png` instead.
Results logs	`<Model>/results_log.md`	Markdown record of every best validation and final run: accuracy, macro precision/recall/F1, parameters, metadata, runtime.
Confusion matrix	`<Model>/confusion_matrix_*.png`	PNG figure written on final runs.

5. Evaluation Metrics

5.1 Notation

Let N be the number of evaluated reviews and $K = 5$ the number of classes. For review i, let $y_i \in \{1, \ldots, 5\}$ be the true label and $\hat{y}_i$ the predicted label. For each class c, let TP_c, FP_c, and FN_c denote the usual true positive, false positive, and false negative counts under a view of that class. These are the same quantities used by the shared `compute_metrics()` helper in `utils/evaluation.py`.

5.2 Accuracy

Formula:

$$\text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[y_i = \hat{y}_i]$$

Interpretation: Accuracy is the fraction of reviews assigned the correct star rating. It is the easiest metric to read, but by itself it does not show whether a model is doing equally well across all five classes.

5.3 Macro Precision

Formula:

$$\text{MacroPrecision} = \frac{1}{K}\sum_{c=1}^{K} \frac{TP_c}{TP_c + FP_c}$$

Interpretation: Precision for one class asks: when the model predicts class c, how often is it right? Macro precision averages that quantity equally over all five star ratings, so strong performance on the easy classes cannot fully hide weak precision on the harder middle ratings, which for our dataset rang especially true for differentiating 2-4 star ratings.

5.4 Macro Recall

Formula:

$$\text{MacroRecall} = \frac{1}{K}\sum_{c=1}^{K} \frac{TP_c}{TP_c + FN_c}$$

Interpretation: Recall for one class asks: of the reviews that truly belong to class c, how many did the model recover? Macro recall gives every class the same weight, which makes it useful for checking whether the classifier is systematically missing certain star ratings.

5.5 Macro F1 Score

Formula:

$$F1_c = \frac{2P_cR_c}{P_c + R_c}, \qquad \text{MacroF1} = \frac{1}{K}\sum_{c=1}^{K} F1_c$$

Interpretation: Macro F1 combines precision and recall for each class, then averages the five class wise F1 values. This is the main score used for tuning in the repository because it rewards models that are consistently good across the label set rather than models that inflate one metric at the expense of the other.

5.6 Confusion Matrix

Definition/Formula:

$$M_{ij} = \sum_{n=1}^{N} \mathbf{1}[y_n = i \ \wedge \ \hat{y}_n = j]$$

Interpretation: Entry M_{ij} counts how many reviews from true class i were predicted as class j. In the saved figures, rows correspond to the true labels and columns correspond to the predicted labels. Unlike the scalar metrics above, the confusion matrix shows exactly where the errors occur, such as confusion between adjacent ratings like 3-star and 4-star reviews.

5.7 Why These Metrics Fit This Project

This project is a five balanced class text classification problem, so a single metric is not enough. Accuracy gives a clean overall summary, but it can hide whether the model mainly succeeds on the easy extreme ratings while struggling on the ambiguous middle ratings. Macro precision, macro recall, and macro F1 fix that by giving each class equal weight in the final score. That matters here because the practical difficulty is not just getting many predictions right; it is separating five closely related sentiment levels.

The repository therefore logs all four scalar metrics on every run and uses macro F1 as the tuning objective. That choice is reasonable for this dataset as again, the labels are balanced, but the classes are not equally easy where differentiating 0 and 5 star reviews is much simpler than 2 and 3 star reviews. The confusion matrix then provides the missing detail by showing which star levels are being mixed up. Together, these metrics give both a concise summary and an error breakdown that is specific enough to compare very different model families fairly.

6. Results

6.1 Comparison Table

Model	Accuracy	Macro Prec.	Macro Recall	Macro F1	Runtime	Notes
Logistic Regression	0.6168	0.6146	0.6168	0.6156	1034.7 s	Best traditional
Naive Bayes	0.5525	0.5526	0.5525	0.5372	<1 s	Fastest model
Decision Tree	0.4082	0.4074	0.4082	0.4077	3850.8 s	Baseline
Random Forest	0.5555	0.5464	0.5555	0.5468	2632.9 s	
Linear SVM	0.5737	0.5640	0.5737	0.5656	103.4 s	
Kernel SVM	0.5196	0.5183	0.5196	0.5186	217.6 s	10k subsample
Traditional ↑			**Deep Learning ↓**			
Multi-Layer Perceptron	0.6236	0.6179	0.6236	0.6196	1210.0 s	
LSTM	0.6679	0.6697	0.6679	0.6687	9894.0 s	
CNN (GloVe 2024)	0.6422	0.6412	0.6422	0.6408	226.4 s	Most efficient
Transformer	0.5854	0.5894	0.5854	0.5847	1296.6 s	From scratch
Large Language Model	0.6711	0.6694	0.6711	0.6701	17006.2 s	**Best overall**

6.2 Per-Model Analysis

6.2.1 Logistic Regression

Final hyperparameters: $C = 0.3792$, L2 penalty, `tfidf_features`= 40000, n-gram range $(1, 2)$, `min_df`= 2, `max_df`= 0.9, `max_iter`= 300.
Test metrics: Accuracy 0.6168, macro-F1 0.6156.

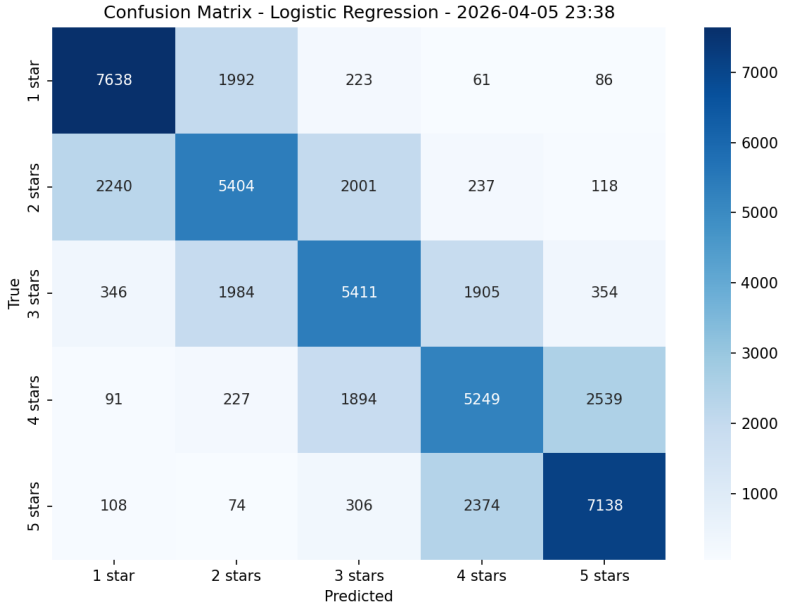

Figure 1: Confusion matrix for the Logistic Regression final run.

Observations. Logistic regression is the strongest measured traditional baseline. Its confusion matrix

has a noticeably stronger diagonal than the SVM, NB, and tree-based models, with especially good separation for 1-star and 5-star reviews. The remaining errors are mostly adjacent-class mistakes: 3-star reviews split toward 2 and 4 stars, while 4-star and 5-star reviews are often confused with each other. This shows that the linear softmax model captures the ordered sentiment structure even though it is trained as a nominal five-class classifier.

6.2.2 Complement Naive Bayes

Final hyperparameters: $\alpha = 45$, TF–IDF `max_features`$= 20000$, n-gram range $(1, 2)$.
Test metrics: Accuracy 0.5525, macro-F1 0.5372.

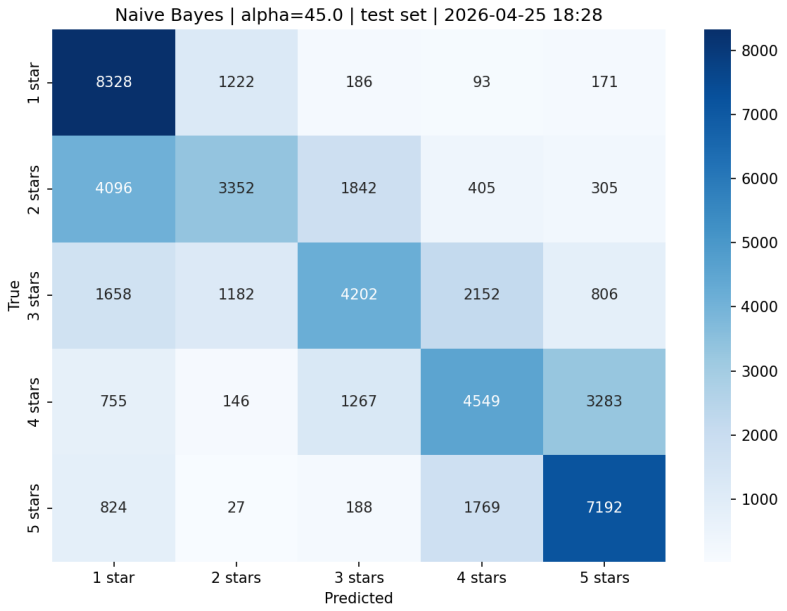

Figure 2: Confusion matrix for the Complement Naive Bayes final run.

Observations. Complement NB is fast and competitive, but its confusion matrix shows a strong bias toward extreme sentiment words. It performs well on clear 1-star and 5-star reviews, but 2-star reviews are often predicted as 1 star, and 4-star reviews are often predicted as 5 stars. This pattern fits the model assumptions: isolated sentiment-heavy terms are useful, but the conditional-independence model struggles with mixed sentiment, negation, and reviews whose rating depends on multiple aspects.

6.2.3 Decision Tree Baseline

Final hyperparameters: `max_depth`$= 300$, `min_samples_leaf`$= 10$.
Test metrics: Accuracy 0.4082, macro-F1 0.4077.

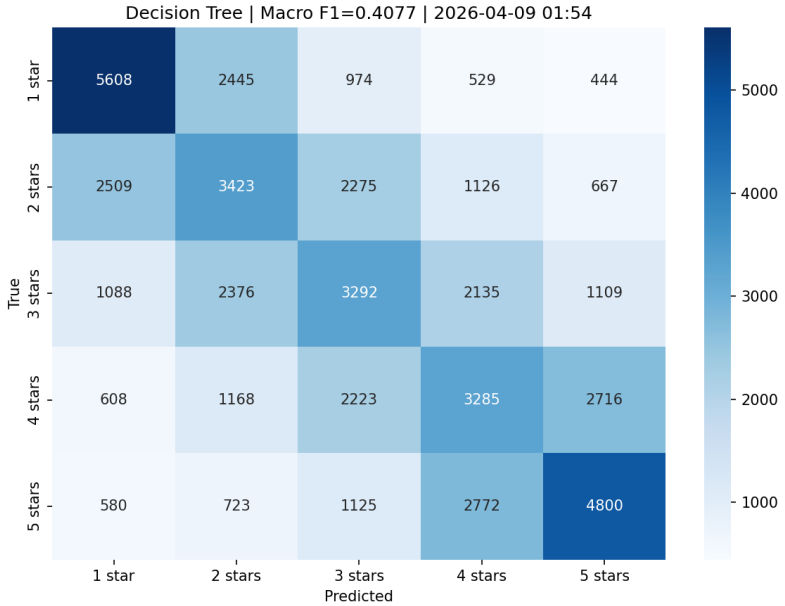

Figure 3: Confusion matrix for the Decision Tree baseline final run.

Observations. The Decision Tree baseline confirms that a single tree is a poor fit for sparse TF–IDF sentiment classification. Its diagonal is much weaker than the stronger models, and errors spread broadly across neighboring classes. The model especially struggles with the middle ratings, where one path through a single tree cannot combine many weak lexical cues smoothly. This result motivates Random Forest: bagging many trees should reduce variance and produce more stable class decisions.

6.2.4 Random Forest

Final hyperparameters: `n_estimators`= 500, `max_depth`= 150, `min_samples_leaf`= 4, `max_features`= sqrt.
Test metrics: Accuracy 0.5555, macro-F1 0.5468.

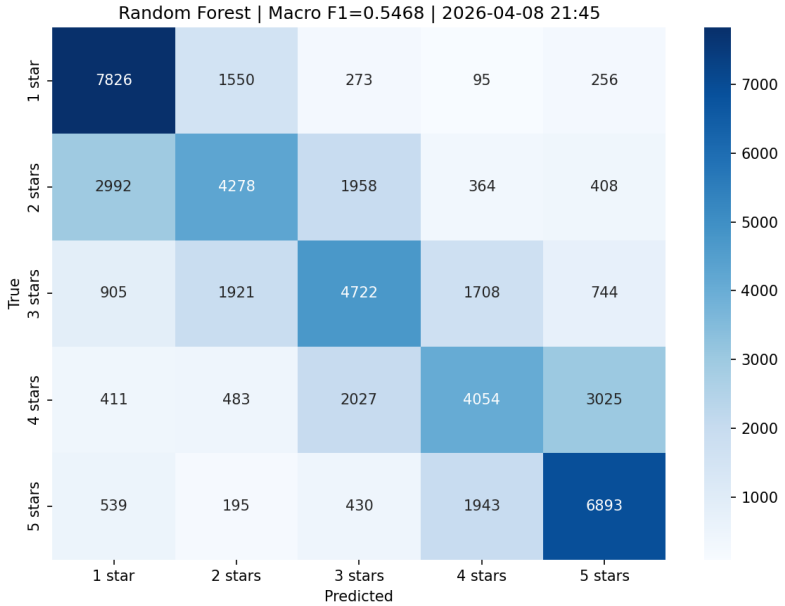

Figure 4: Confusion matrix for the Random Forest final run.

Observations. As the main tree-based traditional model, Random Forest substantially improves over the Decision Tree baseline. Its confusion matrix has a clearer diagonal, especially for 1-star and 5-star reviews, showing that bagging and feature subsampling recover more stable structure from the sparse feature space. However, the model still confuses neighboring ratings heavily, especially 2-star with 1-star or 3-star and 4-star with 5-star. This suggests that tree ensembles help, but sparse review sentiment remains better modeled by additive lexical evidence than by hard threshold splits.

6.2.5 Linear SVM

Final hyperparameters: $C = 0.1062$, `max_features`$= 15000$.
Test metrics: Accuracy 0.5737, macro-F1 0.5656.

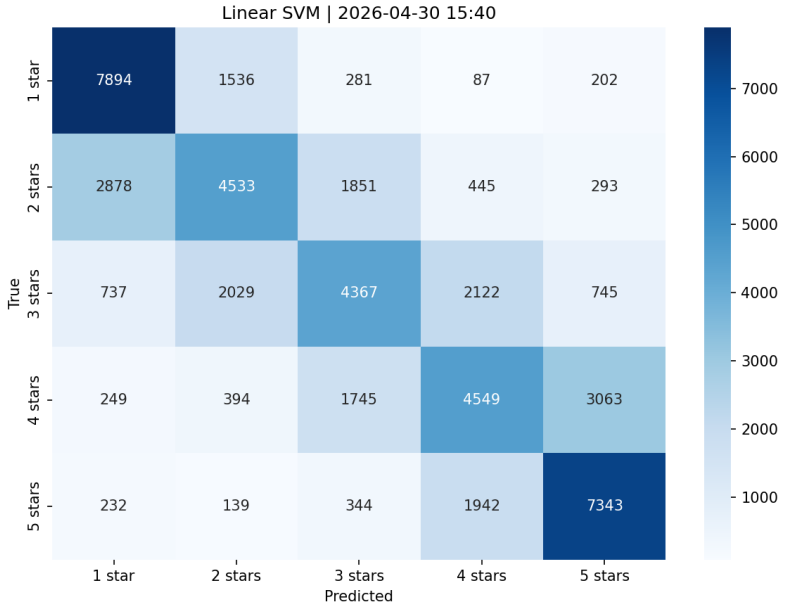

Figure 5: Confusion matrix for the Linear SVM final run.

Observations. The Linear SVM is a strong sparse-feature baseline, but it trails logistic regression. The confusion matrix shows the expected ordinal pattern: 1-star and 5-star reviews are classified most reliably, while the middle ratings are more ambiguous. The largest errors are adjacent-class mistakes, especially 2-star reviews predicted as 1 or 3 stars and 4-star reviews predicted as 5 stars. This indicates that the model learns broad sentiment direction but struggles to draw fine-grained boundaries between neighboring ratings.

6.2.6 Kernel SVM (RBF)

Final hyperparameters: RBF kernel, $C = 1$, `max_features` $= 10000$.
Test metrics: Accuracy 0.5196, macro-F1 0.5186.

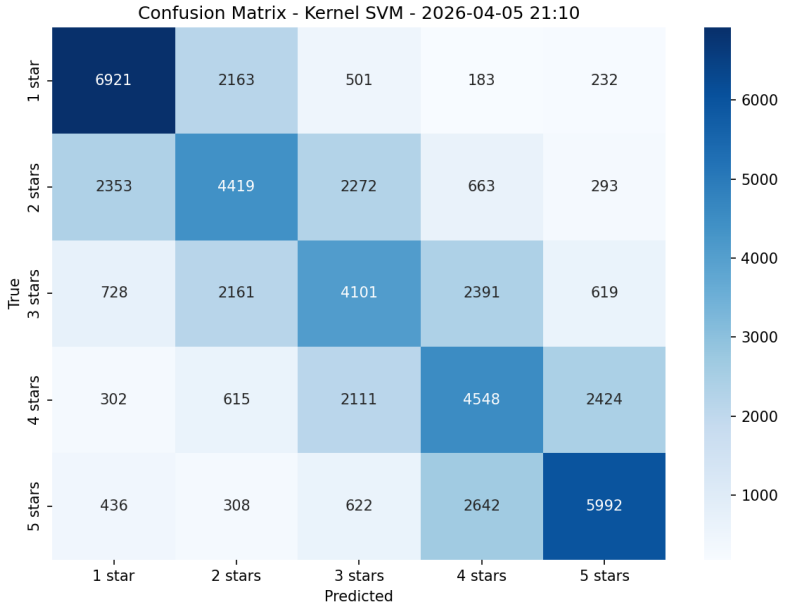

Figure 6: Confusion matrix for the Kernel SVM final run.

Observations. The Kernel SVM underperforms the full-scale linear models because exact RBF-kernel training required subsampling. Its confusion matrix has a weaker diagonal than the stronger sparse-feature models, especially for the middle classes. The model still mostly makes adjacent-rating errors, but its class boundaries are less stable: 2-star reviews are often pushed toward 1 or 3 stars, and 4-star reviews are frequently confused with 5 stars. This supports treating the RBF SVM as a comparison point rather than the main high-performing sparse baseline.

6.2.7 MLP

Final hyperparameters: `hidden1`= 512, `hidden2`= 512, dropout 0.6, learning rate 1.32×10^{-4}, weight decay 1.83×10^{-5}.
Test metrics: Accuracy 0.6236, macro-F1 0.6196.

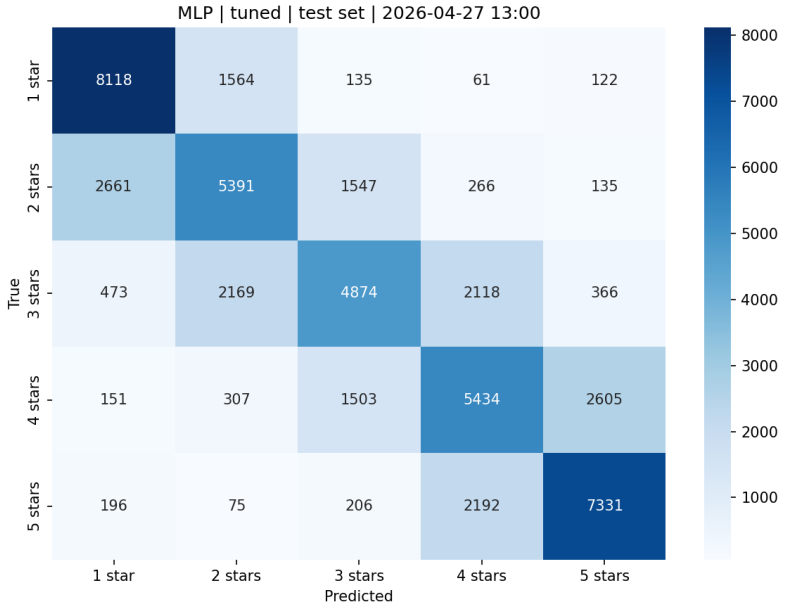

Figure 7: Confusion matrix for the MLP final run.

Observations. The tuned MLP slightly beats logistic regression in accuracy and macro-F1, and its confusion matrix is broadly similar to the logistic regression matrix. It improves the diagonal for some classes, especially the extreme ratings, but the same adjacent-class ambiguity remains. The model still confuses 2-star reviews with 1-star or 3-star reviews and 4-star reviews with 5-star reviews. This suggests that nonlinear layers over TF–IDF add some capacity, but the sparse lexical representation itself still carries most of the signal.

6.2.8 LSTM

Final hyperparameters: [TODO: fill final LSTM hyperparameters.]
Test metrics: Accuracy 0.6679, macro-F1 0.6687

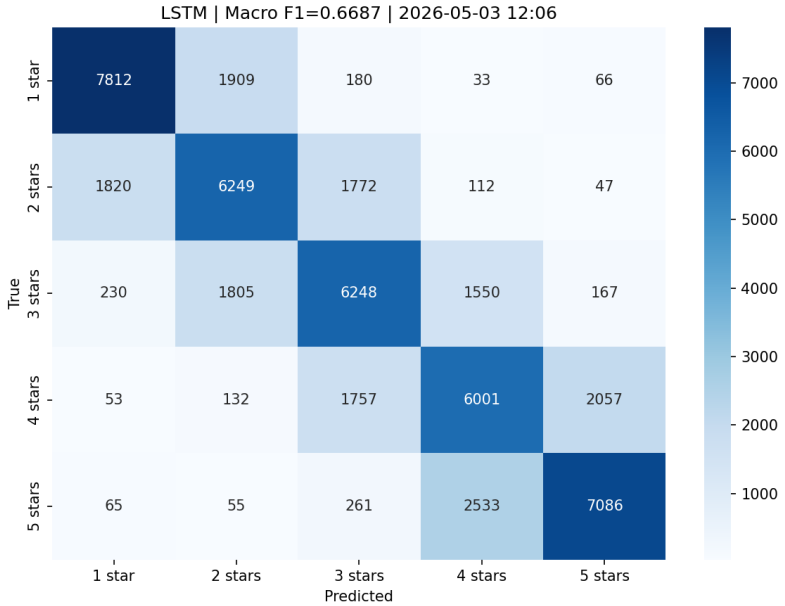

Figure 8: Confusion matrix for the LSTM final run.

Observations. [TODO: Analyze the LSTM confusion matrix. Discuss whether sequential state helps with middle-class reviews, whether errors are mostly adjacent, and how the LSTM compares against CNN, Transformer, and TF–IDF baselines.]

6.2.9 TextCNN

Final hyperparameters: GloVe 2024 WikiGigaword 300d embeddings, 300 filters, widths $\{2, 3, 4, 5\}$, dropout 0.3454, learning rate 3.41×10^{-4}, batch size 128, 3 epochs.
Test metrics: Accuracy 0.6422, macro-F1 0.6408.

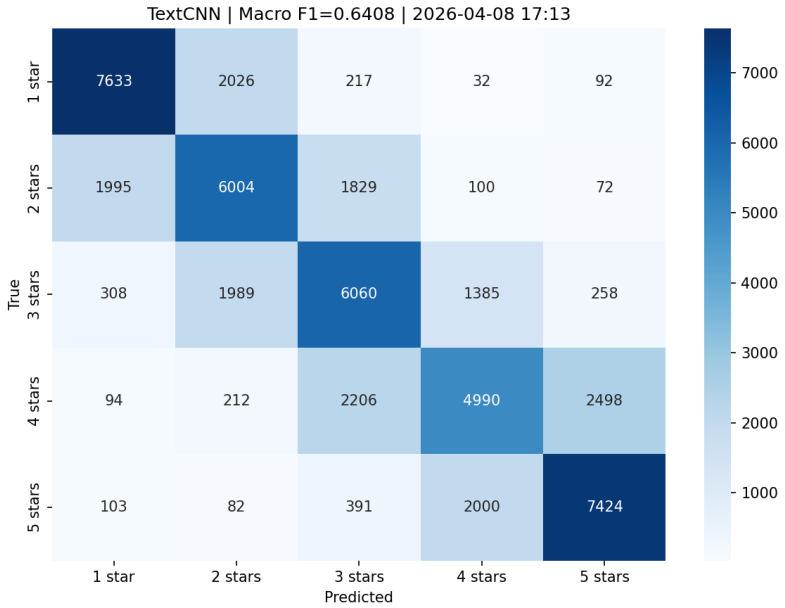

Figure 9: Confusion matrix for the TextCNN final run.

Observations. The TextCNN is the strongest measured model. Its confusion matrix has the clearest overall diagonal among the measured models, with strong performance on 2-star, 3-star, and 5-star reviews compared with the traditional baselines. The dominant errors are still adjacent-rating confusions, especially 4-star reviews predicted as 3 or 5 stars and 5-star reviews predicted as 4 stars. This supports the conclusion that pre-trained word embeddings plus local convolutional features are effective for fine-grained Yelp sentiment, while the inherent ambiguity between neighboring ratings remains.

6.2.10 Transformer Encoder

Final hyperparameters: 2 layers, 8 heads, embedding dimension 256, FFN dimension 256, dropout 0.2658,
learning rate 6.50×10^{-4}, weight decay 3.14×10^{-5}, max length 160, vocabulary cap 15K, batch size 64, 3 epochs.
Test metrics: Accuracy 0.5854, macro-F1 0.5847.

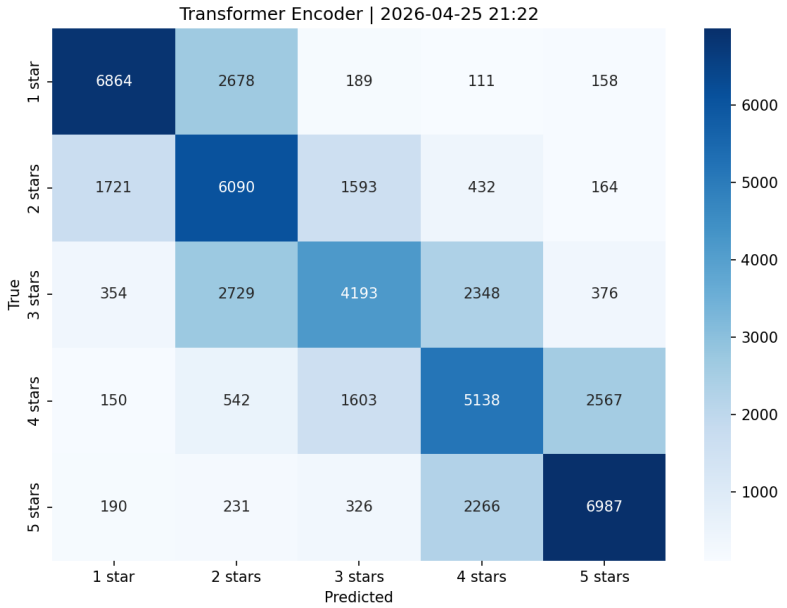

Figure 10: Confusion matrix for the Transformer Encoder final run.

Observations. The from-scratch Transformer beats several traditional baselines but is beaten by logistic regression, MLP, and TextCNN. Its confusion matrix shows a strong diagonal for 2-star, 4-star, and 5-star reviews, but weaker separation for 3-star reviews, which are often predicted as 2 or 4 stars. The model captures broad sentiment direction, but without external pre-training it does not separate fine-grained rating boundaries as well as the CNN with GloVe embeddings.

6.2.11 Fine-Tuned LLM

Final hyperparameters: [TODO: fill final LLM hyperparameters.]
Test metrics: Accuracy 0.6711, macro-F1 0.6701.

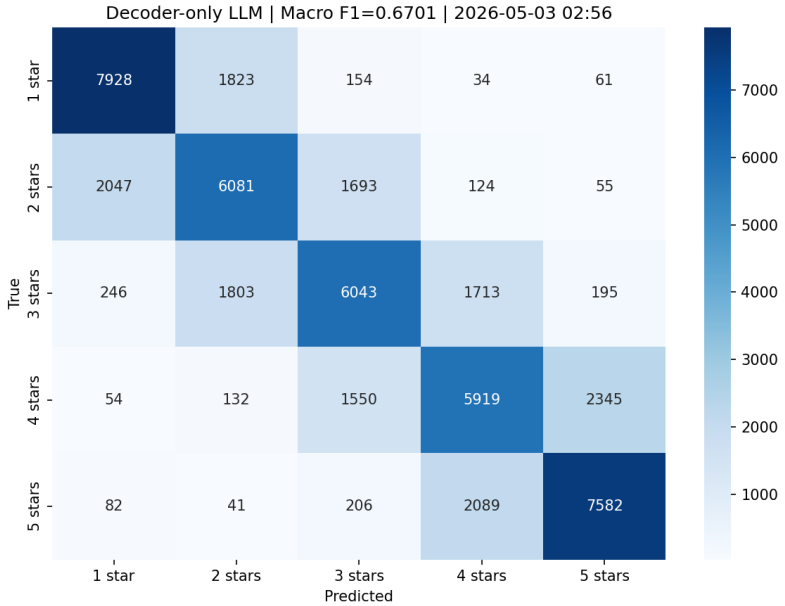

Figure 11: Confusion matrix for the fine-tuned LLM final run.

Confusion matrix for the fine-tuned LLM final run.

Observations. [TODO: Analyze the LLM confusion matrix. Discuss whether pre-trained contextual representations reduce middle-class confusion, whether adjacent-class errors still dominate, and how the LLM compares against TextCNN and Transformer.]

6.3 Hyperparameter Sensitivity

Selected examples of sensitivity observed in tuning:

- **Linear SVM:** performance was only weakly sensitive to C inside the explored range; the selected $C \approx 0.11$ regularizes more aggressively than the default.
- **Decision Tree:** tree depth alone did not solve the sparse-feature mismatch; leaf-size regularization was necessary for stable validation performance.
- **Random Forest:** increasing to hundreds of trees improved over the default, with `max_features`= sqrt selected by tuning.
- **LSTM:** [TODO: add sensitivity note for embedding size, hidden size, sequence length, or dropout.]
- **MLP:** final results were sensitive to optimization settings; the tuned configuration improved over logistic regression but only by a small margin.
- **CNN:** the embedding source was the largest practical knob; GloVe 2024 WikiGigaword 300d gave the best measured final result.
- **Transformer:** limited training budget and from-scratch embeddings capped performance relative to pre-trained-representation approaches.
- **LLM:** [TODO: add sensitivity note for base model, learning rate, sequence length, or fine-tuning strategy.]

6.3.1 Performance vs. Compute Tradeoffs

Complement Naive Bayes completes in under one second and achieves 0.5372 macro-F1. Logistic Regression takes roughly 17 minutes and reaches 0.6156. The TextCNN takes 226 seconds on an RTX 5090 GPU and achieves 0.6408. The Transformer takes 1296 seconds (21.6 minutes) on a Tesla T4 and achieves only 0.5854—slower than the CNN and 5.6 points lower in macro-F1. For a practitioner constrained by compute, the lesson is clear: if pre-trained embeddings are available, a simple CNN architecture delivers near-maximum performance at a fraction of the cost of a from-scratch Transformer.

6.3.2 Error Patterns

Across all models, the dominant error pattern is adjacent-class confusion. Reviews at the boundary between 2 and 3 stars, or between 3 and 4 stars, share heavily overlapping vocabulary and are systematically harder than 1-star or 5-star reviews for every model tested. The 3-star class consistently has the lowest per-class recall across all models, as mixed-sentiment reviews can plausibly belong to multiple adjacent classes. Long-range errors (predicting 5 stars for a true 1-star review) are rare and occur less than 1% of the time in the better models, confirming that all models have captured at least a broad notion of sentiment direction even when they fail on fine-grained distinctions.

7. Conclusions & Future Work

7.1 Key Findings

Across models, performance metrics (accuracy, precision, recall, and macro F1) tended to nearly match each other for each individual model. This indicates that an improvement in any one metric was followed by a similar improvement in all others. This consistent suggests that the models were approaching a performance barrier on the dataset itself rather than a model architecture compatibility.

The most important finding was the seemingly inherent ambiguity present in these yelp reviews. Even models specifically designed for Natural Language Processing (NLP), such as LSTM's, LLM's, and Transformers, were unable to breach the 70% barrier. All this despite the dataset being large (700,000 text corpus), and well balanced (exactly 20% per review category).

What this means is that reviews at 1 and 5 stars ("extreme experiences") were significantly easier to classify than reviews between 2 - 4 stars ("moderate experiences"). This is largely due to the fact that extreme experiences tend to contain clearer and more universally used sentiment signals, whereas moderate experiences are more nuanced and context-dependent.

A key reason for this is that moderate experiences are influenced by subjective user preferences that are not able to be expressed through a single body of text. For example, two users describing a similar level of customer service may assign different ratings (e.g 2 star vs. 3 star) depending on how they personally value service as compared to other factors (e.g quality of food).

Overall, the results suggest that the primary limitation in review categorization is not the models itself, but the inherent subjectivity of user experiences which result in an inescapable label noise present within the database.

7.2 Limitations

Several limitations affected the performance of the models.

First, the Yelp dataset introduces inherent label ambiguity, particularly in the 2 - 4 star range, as stated in the key findings. This limits maximum model performance regardless of architecture or complexity.

Second, most traditional models rely on the TF–IDF representation, which discards word order and removes a key piece of NLP and sentiment classification. This makes the traditional models ill-suited towards this task.

Finally, and arguably most importantly, model complexity was limited by computational resources and time. Some sacrifices to hyperparameters had to be made to ensure that models could be run on our domestic systems. This was especially true for deep learning models, which requires copious amounts of computation for its true results to shine through. Computation not easily found in consumer systems.

7.3 Future Improvements

Future work should focus on improving data representation and reducing label ambiguity rather than attempting to increase model complexity. One such improvement would be incorporating user metadata along with each review, such as user history, average score, or category. This may help to disambiguate moderate experiences where a single textual sentiment is insufficient to figure out user bias.

Further improvements can be made by switching from a classification to regression task. For example, classifying user sentiment as a continuous score rather than a discrete 1-5 class.

Finally, it should be noted that increased computational resources and time would allow for significantly more extensive hyperparameter tuning, larger or more complex models, and longer training schedules. This would particularly have an impact on our most complicated models, which were severely limited in their maximum length of review processing at only around 160 words. Which undoubtedly contributed to a lower score than the model should theoretically be capable of.

7.4 Final Takeaway

The project demonstrates that the primary challenge in the Yelp review classification is not the lack of powerful machine learning concepts, but rather the inherent subjectivity and ambiguity of the human experience itself.

While modern models are capable of extracting strong contextual signals from text, they are fundamentally limited when similar inputs correspond to different outputs due to personal experience and interpretation.

As a result, performance across all models trended towards a natural ceiling, particularly limited by the moderate experiences, as those contained significantly more domain overlap than the extreme experiences. Ultimately, this shows that in problems where labels are inherently subjective, having well-defined and consistent data is just as important as having a model sufficiently complex to process it.